Nor is the fact that you can pretend to be anyone when you log in.

I logged in as one of the richest people on the planet (Elon Musk), just because I could. There is currently no code in the application to verify my claim to be Mr. Musk. Imagine if you could just walk up to Elon's bank and say you are him. Of course, the bank would demand that you verify your claim by providing additional information, such as a secret that only Mr. Musk would know. In most cases, this takes the form of a password, at the very least.

To make your login more secure, you need a way to securely capture and store the password for a user. You also need to compare submitted credentials to those you have stored to ensure the person logging in is who they say they are. Ideally, you also need to provide the user with a way to register an account, log out, and reset their password if they forgot it or believe it to be compromised. That is a lot of work to undertake from scratch, especially when ASP.NET Core Identity does most of it for you.

## *9.2 ASP.NET Core Identity*

ASP.NET Core Identity (Identity) is a framework that supports authentication (determining who a visitor is) and authorization (determining what they are allowed to do). The default implementation uses EF Core to store user details, including their credentials in a database. It also provides a customizable UI for many common scenarios, including registering users; logging them in; resetting passwords; managing their profiles; generating account validation tokens; working with external authentication providers, such as Google, Microsoft, Facebook, Twitter; and more.

Identity is available as NuGet packages. When you first create a web application using the new project wizard in Visual Studio, you can configure it to include Identity at the outset by specifying Individual Accounts as the authentication type (figure 9.3).